Indexing

```
acquire
if(done)
   release(next)
   break
work
release(next)
```

Custom Input from user:

```
Scanner sc = new Scanner(System.in);
Integer bufferSize = sc.nextInt();
Integer n = sc.nextInt();
```

```
nextLine()
nextDouble()
nextFloat()
nextLong()
nextBoolean()
```

Multithreading and concurrency:

1. Print Even Odd with 2 threads
2. Bounded Blocking Queue (Producer Consumer)
3. Producer–Consumer using `BlockingQueue`.
4. Thread-safe Singleton (double-checked locking).
5. Implement a custom thread pool (basic).
6. Fibonacci Using 2 Threads with Semaphore
7. Print ABC Using 3 Threads in Sequence
8. Dining Philosophers
9. Print 1-100 with 3 threads
10. FizzBuzz Multithreaded

Prepare these **first**:

1. Producer Consumer (custom blocking queue)
2. Bounded Blocking Queue
3. Print Even Odd (2 threads)
4. Dining Philosophers
5. FizzBuzz Multithreaded
6. Print ABC with 3 threads

ARYAN:

Print Zero Even Odd
Fizz Buzz Multithreaded
Design Bounded Blocking Queue
The Dining Philosophers
Multithreaded Web Crawler

Print Odd and even Numbers

Print numbers from 1 to N where one thread prints odd numbers and another prints even numbers.

```
class OddEvenPrinter {
    private int number = 1;
    private final int limit;

    public OddEvenPrinter(int limit) {
        this.limit = limit;
    }

    public synchronized void printOdd() throws InterruptedException {
        while (number <= limit) {
            while (number % 2 == 0) {
                wait();
            }
            if (number <= limit) {
                System.out.println("Odd: " + number++);
                notify();
            }
        }
    }

    public synchronized void printEven() throws InterruptedException {
        while (number <= limit) {
            while (number % 2 == 1) {
                wait();
            }
            if (number <= limit) {
                System.out.println("Even: " + number++);
                notify();
            }
        }
    }
}



public class Main {
    public static void main(String[] args) {
        OddEvenPrinter printer = new OddEvenPrinter(10);

        Thread oddThread = new Thread(() -> {
            try {
```

```
                printer.printOdd();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread evenThread = new Thread(() -> {
            try {
                printer.printEven();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        oddThread.start();
        evenThread.start();
    }
}
```

Using Semaphore :

```
import java.util.concurrent.Semaphore;

class OddEvenPrinter {

    private int number = 1;
    private final int limit;

    private Semaphore oddSemaphore = new Semaphore(1);
    private Semaphore evenSemaphore = new Semaphore(0);

    public OddEvenPrinter(int limit) {
        this.limit = limit;
    }

    public void printOdd() throws InterruptedException {

        while (true) {
            oddSemaphore.acquire();
            if (number <= limit) {
                System.out.println("Odd: " + number++);
            }
            evenSemaphore.release();
        }
    }

    public void printEven() throws InterruptedException {

        while (true) {
            evenSemaphore.acquire();
            if (number <= limit) {
                System.out.println("Even: " + number++);
            }

            oddSemaphore.release();
        }
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        OddEvenPrinter printer = new OddEvenPrinter(10);

        Thread oddThread = new Thread(() -> {
            try {
                printer.printOdd();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread evenThread = new Thread(() -> {
            try {
                printer.printEven();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        oddThread.start();
        evenThread.start();
    }
}
```

Prefer way:

```
import java.util.concurrent.Semaphore;

class NumberPrinter {

    private int curr = 1;
    private int n;

    private Semaphore s1 = new Semaphore(1);
    private Semaphore s2 = new Semaphore(0);

    NumberPrinter(int n) {
        this.n = n;
    }

    public void print1() throws InterruptedException {
        while (true) {
            s1.acquire();
            if (curr > n) {
                s2.release();
                break;
            }
            System.out.println("Thread1 : " + curr++);
            s2.release();
        }
    }

    public void print2() throws InterruptedException {
        while (true) {
            s2.acquire();
            if (curr > n) {
                s1.release();
                break;
            }
            System.out.println("Thread2 : " + curr++);
            s1.release();
        }
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        NumberPrinter printer = new NumberPrinter(100);

        Thread t1 = new Thread(() -> {
            try {
                printer.print1();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        Thread t2 = new Thread(() -> {
            try {
                printer.print2();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        t1.start();
        t2.start();
    }
}
```

Print 1-100 with 3 threads

Print 1-100 with 3 threads

```
class NumberPrinter {
    private int number = 1;
    private int limit;

    public NumberPrinter(int limit) {
        this.limit = limit;
    }

    public synchronized void print(int threadId) throws InterruptedException {

        while (number <= limit) {
            while ((number - 1) % 3 != threadId) {
                wait();
            }

            if (number <= limit) {
                System.out.println(Thread.currentThread().getName() + " : " + number);
                number++;
            }

            notifyAll();
        }
    }
}

public class Main {
    public static void main(String[] args) {
     NumberPrinter printer = new NumberPrinter(100);

        Thread t1 = new Thread(() -> {
            try {
                printer.print(0);
            } catch (InterruptedException e) {}
        }, "Thread-1");

        Thread t2 = new Thread(() -> {
```

```
            try {
                printer.print(1);
            } catch (InterruptedException e) {}
        }, "Thread-2");

        Thread t3 = new Thread(() -> {
            try {
                printer.print(2);
            } catch (InterruptedException e) {}
        }, "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Print 1-100 with 3 threads using semaphore

```
import java.util.concurrent.Semaphore;

class NumberPrinter {
    private int number = 1;
    private int limit;
    private Semaphore s1 = new Semaphore(1);
    private Semaphore s2 = new Semaphore(0);
    private Semaphore s3 = new Semaphore(0);

    public NumberPrinter(int limit) {
        this.limit = limit;
    }

    public void print1() throws InterruptedException {
        while (true) {
            s1.acquire();

            if (number > limit) {
                s2.release();
                break;
            }

            System.out.println(Thread.currentThread().getName() + " : " + number++);
            s2.release();
        }
    }

    public void print2() throws InterruptedException {
        while (true) {
            s2.acquire();

            if (number > limit) {
                s3.release();
                break;
            }

            System.out.println(Thread.currentThread().getName() + " : " + number++);
            s3.release();
```

```
        }
    }

    public void print3() throws InterruptedException {
        while (true) {
            s3.acquire();

            if (number > limit) {
                s1.release();
                break;
            }

            System.out.println(Thread.currentThread().getName() + " : " + number++);
            s1.release();
        }
    }
}

public class Main {

    public static void main(String[] args) {

        NumberPrinter printer = new NumberPrinter(100);

        Thread t1 = new Thread(() -> {
            try { printer.print1(); } catch (Exception e) {}
        }, "Thread-1");

        Thread t2 = new Thread(() -> {
            try { printer.print2(); } catch (Exception e) {}
        }, "Thread-2");

        Thread t3 = new Thread(() -> {
            try { printer.print3(); } catch (Exception e) {}
        }, "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }
```

```
}
```

Fibonacci Using 2 Threads (Alternate Printing):

Fibonacci Using 2 Threads (Alternate Printing):

Idea

- Shared Fibonacci state: a, b
- Two threads print numbers **alternately**
- Use a boolean flag to control turn.

```
class FibonacciPrinter {

    private int n;
    private int a = 0;
    private int b = 1;
    private boolean firstThreadTurn = true;

    public FibonacciPrinter(int n) {
        this.n = n;
    }

    public synchronized void printByFirstThread() throws InterruptedException {
        for (int i = 0; i < n; i += 2) {

            while (!firstThreadTurn)
                wait();

            System.out.println(Thread.currentThread().getName() + " : " + a);

            int next = a + b;
            a = b;
            b = next;

            firstThreadTurn = false;
            notify();
        }
    }

    public synchronized void printBySecondThread() throws InterruptedException {
        for (int i = 1; i < n; i += 2) {

            while (firstThreadTurn)
```

```
                wait();

            System.out.println(Thread.currentThread().getName() + " : " + a);

            int next = a + b;
            a = b;
            b = next;

            firstThreadTurn = true;
            notify();
        }
    }
}

public class FibonacciTwoThreads {

    public static void main(String[] args) {

        FibonacciPrinter printer = new FibonacciPrinter(10);

        Thread t1 = new Thread(() -> {
            try {
                printer.printByFirstThread();
            } catch (InterruptedException e) {}
        }, "Thread-1");

        Thread t2 = new Thread(() -> {
            try {
                printer.printBySecondThread();
            } catch (InterruptedException e) {}
        }, "Thread-2");

        t1.start();
        t2.start();
    }
}
```

What Interviewers Check

1. Proper synchronization
2. No race condition
3. Correct Fibonacci update
4. Correct alternation of threads
5. Use of `while(wait)` not `if(wait)`

Possible Follow-up Questions

1. Can we do it with `Lock + Condition`?
Yes.

2. Can we do it with `Semaphore`?
Yes (two semaphores).

3. Why `while` instead of `if` with `wait()`?
To handle **spurious wakeups**.

Fibonacci using 2 threads with Semaphore (cleaner solution)

```
import java.util.concurrent.Semaphore;

class FibonacciPrinter {

    private int a = 0;
    private int b = 1;
    private int limit;

    private Semaphore computeSem = new Semaphore(0);
    private Semaphore printSem = new Semaphore(1);

    FibonacciPrinter(int limit) {
        this.limit = limit;
    }

    public void compute() throws InterruptedException {

        for (int i = 0; i < limit; i++) {
            computeSem.acquire();
            int next = a + b;
            a = b;
            b = next;
            printSem.release();
        }
    }

    public void print() throws InterruptedException {

        for (int i = 0; i < limit; i++) {
            printSem.acquire();
            System.out.println(a);
            computeSem.release();
        }
    }
}

public class Main {
```

```
    public static void main(String[] args) {

        FibonacciPrinter printer = new FibonacciPrinter(10);

        Thread t1 = new Thread(() -> {
            try {
                printer.compute();
            } catch (Exception e) {}
        });

        Thread t2 = new Thread(() -> {
            try {
                printer.print();
            } catch (Exception e) {}
        });

        t1.start();
        t2.start();
    }
}
```

Print ABC Using 3 Threads in Sequence

Print ABC Using 3 Threads in Sequence

Three threads should print A, B, C in order repeatedly.

```
class ABCPrinter {
    private int state = 0;

    public synchronized void printA() throws InterruptedException {
        while (state % 3 != 0) wait();
        System.out.print("A");
        state++;
        notifyAll();
    }

    public synchronized void printB() throws InterruptedException {
        while (state % 3 != 1) wait();
        System.out.print("B");
        state++;
        notifyAll();
    }

    public synchronized void printC() throws InterruptedException {
        while (state % 3 != 2) wait();
        System.out.print("C ");
        state++;
        notifyAll();
    }
}

public class Main {
    public static void main(String[] args) {
        ABCPrinter p = new ABCPrinter();

        new Thread(() -> {
            try { for (int i = 0; i < 5; i++) p.printA(); } catch (Exception e) {}
        }).start();

        new Thread(() -> {
            try { for (int i = 0; i < 5; i++) p.printB(); } catch (Exception e) {}
```

```
        }).start();

        new Thread(() -> {
            try { for (int i = 0; i < 5; i++) p.printC(); } catch (Exception e) {}
        }).start();
    }
}
```

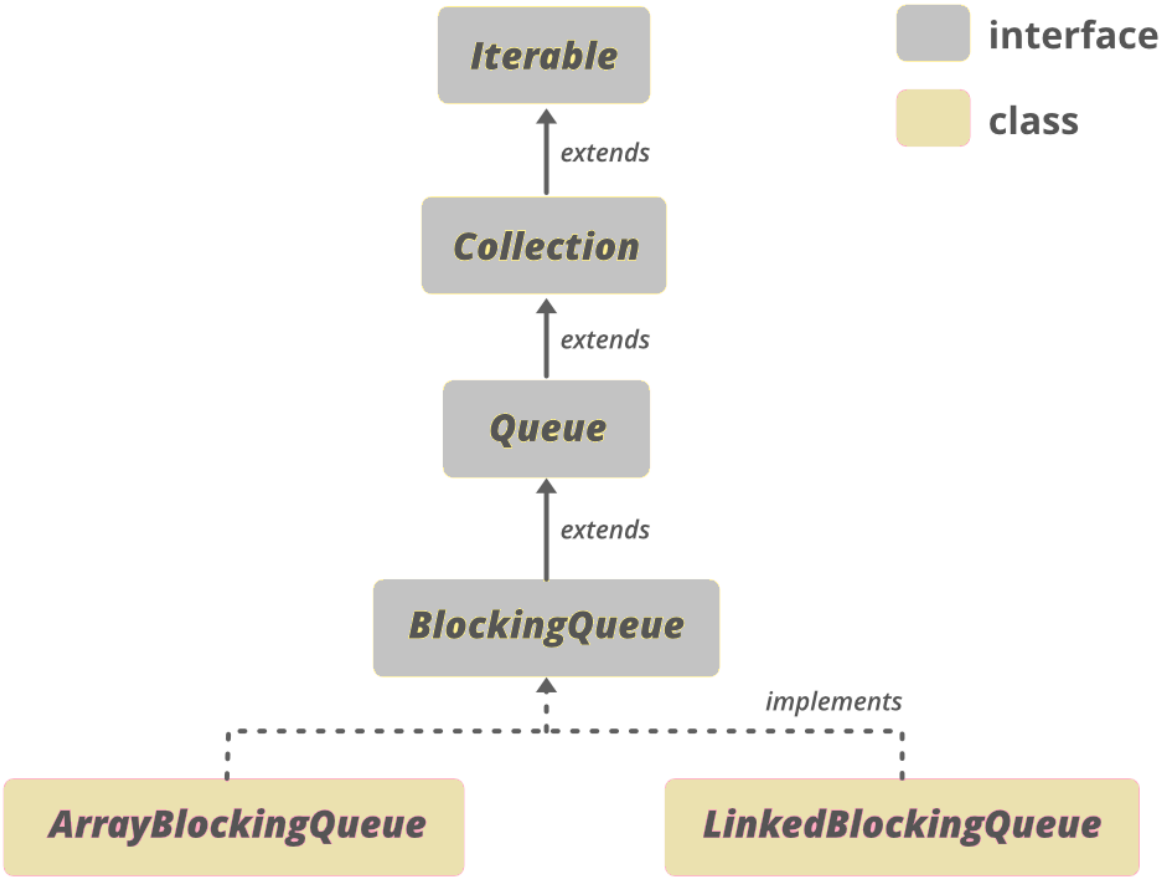
Print ABC Using 3 Threads in Sequence using semaphore

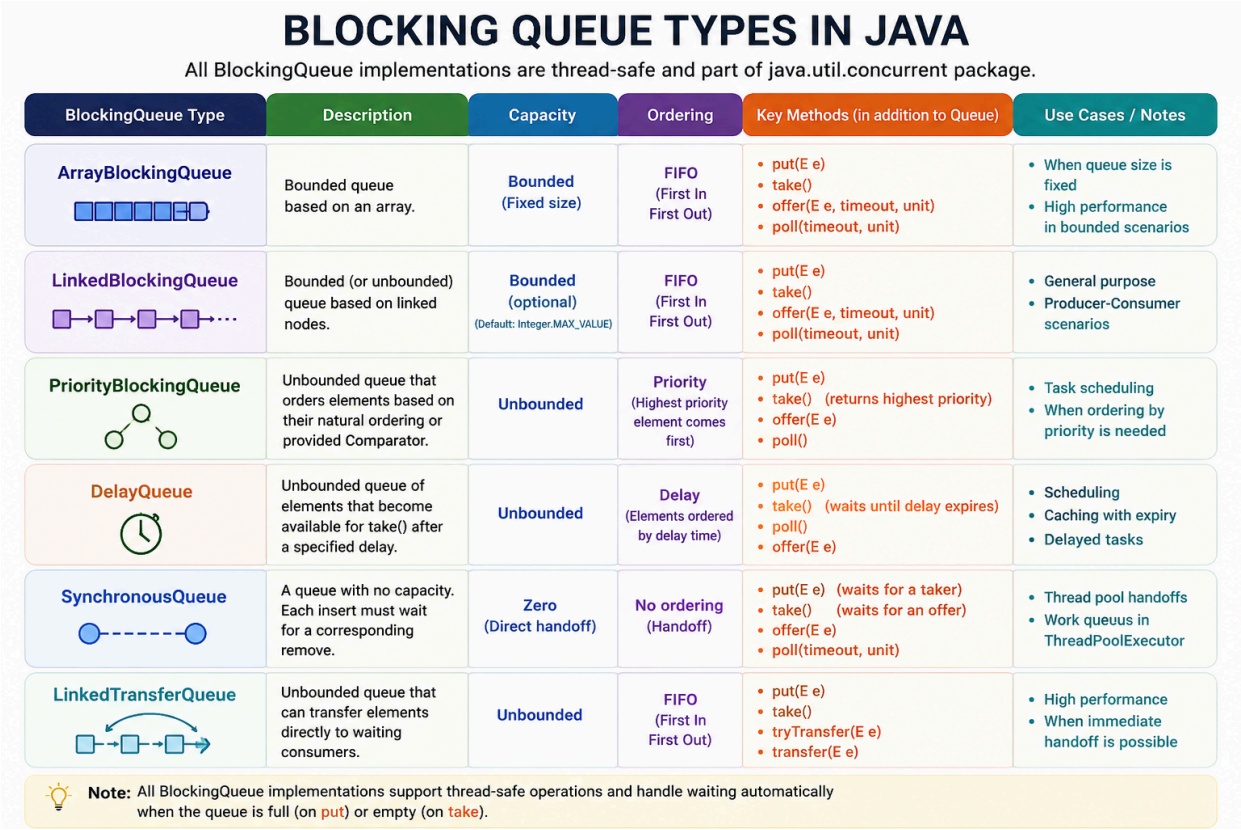
```
import java.util.concurrent.Semaphore;

class ABCPrinter {

    private Semaphore semA = new Semaphore(1);
    private Semaphore semB = new Semaphore(0);
    private Semaphore semC = new Semaphore(0);

    public void printA() throws InterruptedException {
        semA.acquire();
        System.out.print("A");
        semB.release();
    }

    public void printB() throws InterruptedException {
        semB.acquire();
        System.out.print("B");
        semC.release();
    }

    public void printC() throws InterruptedException {
        semC.acquire();
        System.out.print("C ");
        semA.release();
    }
}

public class Main {

    public static void main(String[] args) {

        ABCPrinter printer = new ABCPrinter();
        int n = 5;

        Thread t1 = new Thread(() -> {
            try {
                for (int i = 0; i < n; i++) {
```

```
                printer.printA();
            }
        } catch (Exception e) {}
    });

    Thread t2 = new Thread(() -> {
        try {
            for (int i = 0; i < n; i++) {
                printer.printB();
            }
        } catch (Exception e) {}
    });

    Thread t3 = new Thread(() -> {
        try {
            for (int i = 0; i < n; i++) {
                printer.printC();
            }
        } catch (Exception e) {}
    });

    t1.start();
    t2.start();
    t3.start();
  }
}
```

Bounded Blocking Queue:

Blocking Queue: Blocking queue is an interface.





`BlockingQueue (Interface)` adds:

- `put()`, `take()` → blocking operations
- `offer()`, `poll()` with timeout

2. Class Hierarchy

AbstractQueue<E>
↑
ArrayBlockingQueue<E>
↑
implements BlockingQueue<E>, Serializable

ArrayBlockingQueue<E>

- final Object[] items
- int takeIndex

- int putIndex
- int count
- ReentrantLock lock
- Condition notEmpty
- Condition notFull

+ put(E e)
+ take()
+ offer(E e)
+ poll()

4. Internal Working

- **Uses fixed-size array (circular buffer)**
- **Two pointers:**
  - `putIndex` → **where to insert**
  - `takeIndex` → **where to remove**
- **Synchronization:**
  - **Single** `ReentrantLock`
  - `notFull.await()` → **when queue is full**
  - `notEmpty.await()` → **when queue is empty**

Bounded Blocking Queue:

A **Bounded Blocking Queue** is a thread-safe queue with a **fixed capacity** where:

1. **Producers block** if the queue is **full**.
2. **Consumers block** if the queue is **empty**.

Methods:

Method	Behavior

`put(E e)`	blocks if full
`take()`	blocks if empty
`offer(E e)`	returns false if full
`poll()`	returns null if empty
`peek()`	read without removing
`size()`	current size

But we need to implement enqueue, dequeue, and size;

Key Idea

We maintain three things:

- `availableSlots` → number of empty spaces in queue
- `availableItems` → number of items ready to consume
- `mutex` → ensures only one thread modifies the queue at a time

Code :

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.concurrent.Semaphore;

class BoundedBlockingQueue {

    private Queue<Integer> queue;
    private int capacity;
```

```
    private Semaphore availableSlots;
    private Semaphore availableItems;
    private Semaphore mutex;

    public BoundedBlockingQueue(int capacity) {
        this.capacity = capacity;
        this.queue = new LinkedList<>();

        availableSlots = new Semaphore(capacity);
        availableItems = new Semaphore(0);
        mutex = new Semaphore(1);
    }

    public void enqueue(int element) throws InterruptedException {
        availableSlots.acquire();   // wait if queue is full
       mutex.acquire();
            try {
                    queue.add(element);
                     availableItems.release();   // signal item available
            } finally {
                    mutex.release();
            }
    }
```

```
public int dequeue() throws InterruptedException {
        availableItems.acquire();   // wait if queue empty
        mutex.acquire();
          try {
                int value = queue.poll();
          availableSlots.release();   // signal empty slot
      } finally {
          mutex.release();
      }
        return value;
    }
```

```
public int size() throws InterruptedException {
int size;
    mutex.acquire();
    try {
       size = queue.size();
    } finally {
        mutex.release();
    }
          return size;
}

}
```

Producer consumer:

```
public class Main {

    public static void main(String[] args) {

        BoundedBlockingQueue queue = new BoundedBlockingQueue(3);

        Thread producer = new Thread(() -> {
            try {
                for(int i=1;i<=5;i++){
                    queue.enqueue(i);
```

```
                System.out.println("Produced: " + i + " size=" + queue.size());
            }
        } catch (Exception e) {}
    });

    Thread consumer = new Thread(() -> {
        try {
            for(int i=1;i<=5;i++){
                int val = queue.dequeue();
                 System.out.println("Consumed: " + val + " size=" + queue.size());
            }
        } catch (Exception e) {}
    });

    producer.start();
    consumer.start();
  }
}
```

Q.) Why size() needs a lock?

`size()` needs the lock (`mutex`) because the queue is not thread-safe and may be modified concurrently by `enqueue()` and `dequeue()`.

Without the lock, `size()` could read the queue while another thread is modifying it, which leads to race conditions and inconsistent results.

Creating separate Producer, Consumer Class:

```
class Producer implements Runnable {
   private BoundedBlockingQueue queue;

   Producer(BoundedBlockingQueue queue) {
      this.queue = queue;
   }

   public void run() {

      try {
         for (int i = 1; i <= 5; i++) {
            queue.enqueue(i);
            System.out.println(  "Produced: " + i +  " size=" + queue.size() );
            Thread.sleep(500);
         }
      } catch (Exception e) {
         Thread.currentThread().interrupt();
      }
   }
}
```

Consumer Class:

```
class Consumer implements Runnable {
   private BoundedBlockingQueue queue;

   Consumer(BoundedBlockingQueue queue) {
      this.queue = queue;
   }

   public void run() {

      try {
         for (int i = 1; i <= 5; i++) {
            int val = queue.dequeue();
            System.out.println("Consumed: " + val +  " size=" + queue.size()
            );
```

```
                Thread.sleep(800);
            }
        } catch (Exception e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Main Class:

```
public class Main {

    public static void main(String[] args) {

        BoundedBlockingQueue queue = new BoundedBlockingQueue(3);

        Thread producer = new Thread(new Producer(queue));
        Thread consumer = new Thread(new Consumer(queue));

        producer.start();
        consumer.start();
    }
}
```

Producer–Consumer Using BlockingQueue

Producer Consumer using Blocking Queue:

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class Producer implements Runnable {
    private BlockingQueue<Integer> queue;

    Producer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                queue.put(i);
                System.out.println("Produced: " + i);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

class Consumer implements Runnable {
    private BlockingQueue<Integer> queue;

    Consumer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                int val = queue.take();
                System.out.println("Consumed: " + val);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(2);

        Thread producer = new Thread(new Producer(queue));
        Thread consumer = new Thread(new Consumer(queue));

        producer.start();
        consumer.start();
    }
}
```

Producer consumer using wait and notify:

```
import java.util.*;

class SharedBuffer {

   private Queue<Integer> queue = new LinkedList<>();
   private int capacity;

   SharedBuffer(int capacity) {
      this.capacity = capacity;
   }

   public synchronized void produce(int value) throws InterruptedException {

      while (queue.size() == capacity) {
         wait();   // buffer full
      }

      queue.add(value);
      System.out.println("Produced: " + value);

      notify();  // wake consumer
   }

   public synchronized void consume() throws InterruptedException {

      while (queue.isEmpty()) {
         wait();   // buffer empty
      }

      int value = queue.poll();
      System.out.println("Consumed: " + value);

      notify();  // wake producer
   }
}

public class Main {
   public static void main(String[] args) {
      SharedBuffer buffer = new SharedBuffer(5);
```

```
Thread producer = new Thread(() -> {
        int value = 1;
        try {
            while (true) {
                buffer.produce(value++);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {}
    });

  Thread consumer = new Thread(() -> {
        try {
            while (true) {
                buffer.consume();
                Thread.sleep(800);
            }
        } catch (InterruptedException e) {}
    });

    producer.start();
    consumer.start();
  }
}
```

Read–Write Lock Problem:

Read–Write Lock Problem:

Problem

- Multiple threads can **read simultaneously**
- Only **one thread can write at a time**
- No read during write ❌
- No write during read ❌

Key Concept

Use **ReentrantReadWriteLock**

- `readLock()` → shared lock
- `writeLock()` → exclusive lock

```
import java.util.concurrent.locks.*;

class SharedResource {

    private int data = 0;

    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    // READ operation
    public void read() {
        lock.readLock().lock();
        try {
            System.out.println(Thread.currentThread().getName() +
                    " reading value: " + data);

            Thread.sleep(100); // simulate read delay
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock.readLock().unlock();
        }
    }

    // WRITE operation
    public void write(int value) {
        lock.writeLock().lock();
        try {
            System.out.println(Thread.currentThread().getName() +
                    " writing value: " + value);

            Thread.sleep(100); // simulate write delay
            data = value;

        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            lock.writeLock().unlock();
        }
    }
}

public class Main {

    public static void main(String[] args) {
```

```
        SharedResource resource = new SharedResource();

        // Reader threads
        Runnable reader = () -> {
            for (int i = 0; i < 3; i++) {
                resource.read();
            }
        };

        // Writer thread
        Runnable writer = () -> {
            for (int i = 1; i <= 3; i++) {
                resource.write(i * 10);
            }
        };

        Thread r1 = new Thread(reader, "Reader-1");
        Thread r2 = new Thread(reader, "Reader-2");
        Thread w1 = new Thread(writer, "Writer-1");

        r1.start();
        r2.start();
        w1.start();
    }
}
```

Dining Philosopher Problem

Dining Philosopher Problem:

```
import java.util.concurrent.Semaphore;

class DiningPhilosophers {

private final Semaphore table = new Semaphore(4);
private final Semaphore[] forks = new Semaphore[5];


    public DiningPhilosophers() {
        for (int i = 0; i < 5; i++) {
            forks[i] = new Semaphore(1);
        }
    }

    public void wantsToEat(int id) throws InterruptedException {

        int left = id;
        int right = (id + 1) % 5;

        table.acquire();

        forks[left].acquire();
        forks[right].acquire();

        eat(id);

        forks[left].release();
        forks[right].release();

        table.release();
    }

    private void eat(int id) {
        System.out.println("Philosopher " + id + " is eating");
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        DiningPhilosophers dp = new DiningPhilosophers();

        for (int i = 0; i < 5; i++) {
            int id = i;
            new Thread(() -> {
                try {
                    dp.wantsToEat(id);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
    }
}
```

Another approach:

```
import java.util.concurrent.Semaphore;

class DiningPhilosophers {

    private Semaphore semaphore;
    private Semaphore[] forks;

    public DiningPhilosophers() {
        // Allow only 4 philosophers to try eating simultaneously
        semaphore = new Semaphore(4);

        forks = new Semaphore[5];
        for (int i = 0; i < 5; i++) {
            forks[i] = new Semaphore(1);
        }
    }

    public void wantsToEat(int philosopher,
                           Runnable pickLeftFork,
                           Runnable pickRightFork,
                           Runnable eat,
                           Runnable putLeftFork,
                           Runnable putRightFork) throws InterruptedException {

        semaphore.acquire();

        int left = philosopher;
        int right = (philosopher + 1) % 5;

        Semaphore leftFork = forks[left];
        Semaphore rightFork = forks[right];

        leftFork.acquire();
        rightFork.acquire();

        pickLeftFork.run();
        pickRightFork.run();
```

```
        eat.run();

        putRightFork.run();
        rightFork.release();

        putLeftFork.run();
        leftFork.release();

        semaphore.release();
    }
}

public class Main {

    public static void main(String[] args) {

        DiningPhilosophers dp = new DiningPhilosophers();

        for (int i = 0; i < 5; i++) {

            int philosopher = i;

            new Thread(() -> {
                try {
                    dp.wantsToEat(
                            philosopher,

                            () -> System.out.println("Philosopher " + philosopher + " picked LEFT
fork"),

                            () -> System.out.println("Philosopher " + philosopher + " picked RIGHT
fork"),

                            () -> System.out.println("Philosopher " + philosopher + " is EATING"),

                            () -> System.out.println("Philosopher " + philosopher + " put LEFT
fork"),
```

```
                        () -> System.out.println("Philosopher " + philosopher + " put RIGHT
fork")

                    );
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
    }
}
```

Why this works

- Maximum **4 philosophers** allowed → circular wait breaks
- **Deadlock avoided**

Other Solutions Interviewers Like

1. **Odd-Even Fork Pickup**
2. **ReentrantLock with tryLock**
3. **Waiter solution (monitor)**

Thread-Safe Singleton (Double Checked Locking)

1. Double Checked Locking

Dont forget to add class in singletonholder

Thread-Safe Singleton (Double Checked Locking):

Implement a thread-safe Singleton with lazy initialization.

```
class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

1. Enum Singleton (Best and Recommended)

```
public enum Singleton {
    INSTANCE;

    public void doSomething() {
        System.out.println("Singleton using enum");
    }
}
```

2. Bill Pugh Singleton (Best Lazy Initialization)

This is the **best classic singleton pattern** for lazy initialization.

```
public class Singleton {
  private Singleton() {}

  private static class SingletonHelper {
     private static final Singleton INSTANCE = new Singleton();
  }

  public static Singleton getInstance() {
     return SingletonHelper.INSTANCE;
  }
}
```

Factory:

```
class NotificationFactory {

   public static Notification createNotification(String type) {

      if (type.equalsIgnoreCase("EMAIL")) {
         return new EmailNotification();
      } else if (type.equalsIgnoreCase("SMS")) {
         return new SMSNotification();
      } else if (type.equalsIgnoreCase("PUSH")) {
         return new PushNotification();
      }

      throw new IllegalArgumentException("Invalid type");
   }
}
```

Notification n = Notification.createNotificaiton(“email”);

FizzBuzz:

FizzBuzz:

```
import java.util.concurrent.Semaphore;
import java.util.function.IntConsumer;

class FizzBuzz {

    private int n;

    private final Semaphore numberSemaphore;
    private final Semaphore fizzSemaphore;
    private final Semaphore buzzSemaphore;
    private final Semaphore fizzBuzzSemaphore;

    public FizzBuzz(int n) {
        this.n = n;

        numberSemaphore = new Semaphore(1);
        fizzSemaphore = new Semaphore(0);
        buzzSemaphore = new Semaphore(0);
        fizzBuzzSemaphore = new Semaphore(0);
    }

    public void fizz(Runnable printFizz) throws InterruptedException {
        for (int i = 1; i <= n; i++) {

            if (i % 3 == 0 && i % 5 != 0) {

                fizzSemaphore.acquire();

                printFizz.run();

                numberSemaphore.release();
            }
        }
    }

    public void buzz(Runnable printBuzz) throws InterruptedException {
        for (int i = 1; i <= n; i++) {

            if (i % 5 == 0 && i % 3 != 0) {

                buzzSemaphore.acquire();
```

```
                printBuzz.run();

                numberSemaphore.release();
            }
        }
    }

    public void fizzbuzz(Runnable printFizzBuzz) throws InterruptedException {
        for (int i = 1; i <= n; i++) {

            if (i % 3 == 0 && i % 5 == 0) {

                fizzBuzzSemaphore.acquire();

                printFizzBuzz.run();

                numberSemaphore.release();
            }
        }
    }

    public void number(IntConsumer printNumber) throws InterruptedException {

        for (int i = 1; i <= n; i++) {

            numberSemaphore.acquire();

            if (i % 3 == 0 && i % 5 == 0) {
                fizzBuzzSemaphore.release();
            }
            else if (i % 3 == 0) {
                fizzSemaphore.release();
            }
            else if (i % 5 == 0) {
                buzzSemaphore.release();
            }
            else {
                printNumber.accept(i);
                numberSemaphore.release();
            }
        }
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        int n = 20;

        FizzBuzz fb = new FizzBuzz(n);

        Thread fizzThread = new Thread(() -> {
            try {
                fb.fizz(() -> System.out.print("Fizz "));
            } catch (Exception e) {}
        });

        Thread buzzThread = new Thread(() -> {
            try {
                fb.buzz(() -> System.out.print("Buzz "));
            } catch (Exception e) {}
        });

        Thread fizzBuzzThread = new Thread(() -> {
            try {
                fb.fizzbuzz(() -> System.out.print("FizzBuzz "));
            } catch (Exception e) {}
        });

        Thread numberThread = new Thread(() -> {
            try {
                fb.number((x) -> System.out.print(x + " "));
            } catch (Exception e) {}
        });

        fizzThread.start();
        buzzThread.start();
        fizzBuzzThread.start();
        numberThread.start();
    }
}
```

Deadlock :

Deadlock :

```
class Resource {
    private final String name;

    public Resource(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

public class DeadlockDemo {

    public static void main(String[] args) {

        Resource r1 = new Resource("R1");
        Resource r2 = new Resource("R2");

        Thread t1 = new Thread(() -> {
            synchronized (r1) {
                System.out.println("Thread1 locked " + r1.getName());

                try { Thread.sleep(100); } catch (InterruptedException e) {}

                synchronized (r2) {
                    System.out.println("Thread1 locked " + r2.getName());
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (r2) {
                System.out.println("Thread2 locked " + r2.getName());

                try { Thread.sleep(100); } catch (InterruptedException e) {}

                synchronized (r1) {
```

```
                System.out.println("Thread2 locked " + r1.getName());
            }
        }
    });

    t1.start();
    t2.start();
  }
}
```

Avoid Deadlock :

Always acquire locks in same order

```
class Resource {
   private final String name;

   public Resource(String name) {
      this.name = name;
   }

   public String getName() {
      return name;
   }
}

public class DeadlockFixed {

   public static void main(String[] args) {

      Resource r1 = new Resource("R1");
      Resource r2 = new Resource("R2");

      Thread t1 = new Thread(() -> lockInOrder(r1, r2));
      Thread t2 = new Thread(() -> lockInOrder(r1, r2)); // same order
```

```
        t1.start();
        t2.start();
    }

    public static void lockInOrder(Resource r1, Resource r2) {
        synchronized (r1) {
            System.out.println(Thread.currentThread().getName() + " locked " +
r1.getName());

            try { Thread.sleep(100); } catch (InterruptedException e) {}

            synchronized (r2) {
                System.out.println(Thread.currentThread().getName() + " locked
" + r2.getName());
            }
        }
    }
}
```

Future and Call Problem

Q.) Find the sum of square of even numbers using multithreading

```
import java.util.*;
import java.util.concurrent.*;

class SumTask implements Callable<Long> {

    private List<Integer> list;
    private int start;
    private int end;

    public SumTask(List<Integer> list, int start, int end) {
        this.list = list;
        this.start = start;
        this.end = end;
    }

    @Override
    public Long call() {
        long localSum = 0;

        for (int i = start; i < end; i++) {
            int num = list.get(i);
            if (num % 2 == 0) {
                localSum += num * num;
            }
        }
        return localSum;
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        List<Integer> list = Arrays.asList(1,2,3,4,5,6,7,8,9,10);

        int numThreads = 4;
        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        List<Future<Long>> futures = new ArrayList<>();
        int chunkSize = (list.size() + numThreads - 1) / numThreads;

        // Submit tasks
        for (int i = 0; i < list.size(); i += chunkSize) {
            int start = i;
```

```
            int end = Math.min(i + chunkSize, list.size());
            futures.add(executor.submit(new SumTask(list, start, end)));
        }

        long finalSum = 0;

        // Collect results
        for (Future<Long> future : futures) {
            finalSum += future.get();
        }

        executor.shutdown();

        System.out.println("Sum of squares of even numbers: " + finalSum);
    }
}
```

Main Exceptions from `Future:`

1. `ExecutionException`

- Thrown when **task itself throws an exception**
- Wraps the **actual exception**

```
Future<Integer> future = executor.submit(() -> {
    int x = 10 / 0; // ArithmeticException
    return x;
});

try {
    future.get();
} catch (ExecutionException e) {
    System.out.println(e.getCause()); // ArithmeticException
}
```

Always use: e.getCause()

2. `InterruptedException`

- Thrown when **thread waiting on `get()` is interrupted**

```
try {
   future.get();
} catch (InterruptedException e) {
   Thread.currentThread().interrupt(); // best practice
}
```

---

3. ❌ `CancellationException`

- Thrown when:
  - Task was **cancelled**
  - and you call `get()`

```
future.cancel(true);
future.get(); // ❌ throws CancellationException
```

Summary Table

Exception	When
`ExecutionException`	Task threw exception
`InterruptedException`	Waiting thread interrupted
`CancellationException`	Task cancelled

Important Clarification

👉 `Future` itself does NOT throw exceptions during:

```
executor.submit(...)
```

👉 All exceptions appear during:

future.get()

Interview Insight

Q: Why don’t we see exception immediately?

✔️ Answer:

> Because `submit()` captures exceptions and stores them inside `Future`, to be retrieved later via `get()`

🔥 Difference with `execute()`

Method	Exception Behavior
`submit()`	Wrapped in `Future`
`execute()`	Thrown immediately (uncaught handler)

Sum of Array Using Multithreading

Sum of Array Using Multithreading

✅ Approach

- Split array into chunks
- Each thread computes partial sum
- Combine results using `Future`

```
import java.util.*;
import java.util.concurrent.*;

// Callable Task
class SumTask implements Callable<Long> {
    private int[] arr;
    private int start;
    private int end;

    public SumTask(int[] arr, int start, int end) {
        this.arr = arr;
        this.start = start;
        this.end = end;
    }

    @Override
    public Long call() {
        long sum = 0;
        for (int i = start; i < end; i++) {
            sum += arr[i];
        }
        return sum;
    }
}

public class Main {

    public static void main(String[] args) throws Exception {

        int[] arr = {1,2,3,4,5,6,7,8,9,10};
```

```
        int numThreads = 4;
        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        List<Future<Long>> futures = new ArrayList<>();

        int chunkSize = (arr.length + numThreads - 1) / numThreads;

        // Submit tasks
        for (int i = 0; i < arr.length; i += chunkSize) {
            int start = i;
            int end = Math.min(i + chunkSize, arr.length);
            futures.add(executor.submit(new SumTask(arr, start, end)));
        }

        long totalSum = 0;

        // Collect results
        for (Future<Long> future : futures) {
            totalSum += future.get();
        }

        executor.shutdown();

        System.out.println("Total Sum: " + totalSum);
    }
}
```

Using ThreadPoolExecutor:

```
import java.util.*;
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) throws Exception {

        int[] arr = new int[100];
        Arrays.fill(arr, 1);

        int numThreads = 4;

        ExecutorService executor = new ThreadPoolExecutor(
                numThreads,
                numThreads,
                0L, TimeUnit.MILLISECONDS,
                new ArrayBlockingQueue<>(5),
                new ThreadPoolExecutor.AbortPolicy()
        );

        List<Future<Long>> futures = new ArrayList<>();

        int chunkSize = arr.length / numThreads + (arr.length % numThreads == 0 ? 0 : 1);

        for (int i = 0; i < arr.length; i+=chunkSize) {

            int start = i ;
            int end = Math.min(chunkSize, arr.length);

            if (start >= arr.length) break;

            futures.add(executor.submit(() -> {
                long localSum = 0;
                for (int j = start; j < end; j++) {
                    localSum += arr[j];
                }
                return localSum;
            }));
        }

        long total = 0;
```

```
        for (Future<Long> f : futures) {
            try {
                total += f.get();
            } catch (ExecutionException e) {
                System.out.println("Task failed: " + e.getCause());
            }
        }

        executor.shutdown();

        System.out.println("Total Sum: " + total);
    }
}
```

Using CompletableFuture:

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

public class Main {

    public static void main(String[] args) {

        int[] arr = {1,2,3,4,5,6,7,8,9,10};

        int numThreads = 4;
        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        int chunkSize = (arr.length + numThreads - 1) / numThreads;

        List<CompletableFuture<Long>> futures = new ArrayList<>();

        for (int i = 0; i < arr.length; i += chunkSize) {
            int start = i;
            int end = Math.min(i + chunkSize, arr.length);

            CompletableFuture<Long> future = CompletableFuture.supplyAsync(() -> {
                long sum = 0;
                for (int j = start; j < end; j++) {
                    sum += arr[j];
                }
                return sum;
            }, executor);

            futures.add(future);
        }
```

```
        // Combine results
        long totalSum = futures.stream()
                .map(CompletableFuture::join) // non-checked exception
                .mapToLong(Long::longValue)
                .sum();

        executor.shutdown();

        System.out.println("Total Sum: " + totalSum);
    }
}


import java.util.*;
import java.util.concurrent.*;

// Callable Task (same as yours)
class SumTask implements Callable<Long> {
    private int[] arr;
    private int start;
    private int end;

    public SumTask(int[] arr, int start, int end) {
        this.arr = arr;
        this.start = start;
        this.end = end;
    }

    @Override
    public Long call() {
        long sum = 0;
        for (int i = start; i < end; i++) {
            sum += arr[i];
        }
        return sum;
    }
}

public class Main {

    public static void main(String[] args) {

        int[] arr = {1,2,3,4,5,6,7,8,9,10};
```

```
        int numThreads = 4;
        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        List<CompletableFuture<Long>> futures = new ArrayList<>();

        int chunkSize = (arr.length + numThreads - 1) / numThreads;

        // Submit tasks using SumTask explicitly
        for (int i = 0; i < arr.length; i += chunkSize) {
            int start = i;
            int end = Math.min(i + chunkSize, arr.length);

            SumTask task = new SumTask(arr, start, end);

            CompletableFuture<Long> future = CompletableFuture.supplyAsync(() -> {
                try {
                    return task.call(); // reuse Callable
                } catch (Exception e) {
                    throw new RuntimeException(e);
                }
            }, executor);

            futures.add(future);
        }

        // Combine results
        CompletableFuture<Long> finalResult =
                CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))
                        .thenApply(v ->
                                futures.stream()
                                        .mapToLong(CompletableFuture::join)
                                        .sum()
                        );

        long totalSum = finalResult.join();

        executor.shutdown();

        System.out.println("Total Sum: " + totalSum);
    }
}
```

Bonus Insight (Very Important)

Even with `CompletableFuture`:

future.join();

You are still **waiting at the end**

So:

- It becomes async **internally**
- But final aggregation is still **blocking**

1. `CompletableFuture.allOf()`

✅ Problem

You have multiple async tasks → want to wait for all of them

❌ Without `allOf()`

```
for (CompletableFuture<Long> f : futures) {

   total += f.join(); // waiting one by one

}
```

Not elegant / harder to compose

✅ **With `allOf()`**

```
import java.util.*;
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) {

        int[] arr = {1,2,3,4,5,6,7,8,9,10};
        int numThreads = 4;

        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        List<CompletableFuture<Long>> futures = new ArrayList<>();

        int chunkSize = (arr.length + numThreads - 1) / numThreads;

        for (int i = 0; i < arr.length; i += chunkSize) {
            int start = i;
            int end = Math.min(i + chunkSize, arr.length);

            futures.add(CompletableFuture.supplyAsync(() -> {
                long sum = 0;
                for (int j = start; j < end; j++) {
                    sum += arr[j];
                }
                return sum;
            }, executor));
        }

        // ✅ Wait for all
```

```
        CompletableFuture<Void> all = CompletableFuture.allOf(
                futures.toArray(new CompletableFuture[0])
        );

        // ✅ Combine after all complete
        long total = all.thenApply(v ->
                futures.stream()
                        .mapToLong(CompletableFuture::join)
                        .sum()
        ).join();

        executor.shutdown();

        System.out.println("Total Sum: " + total);
    }
}
```

🔍 Key Idea

CompletableFuture.allOf(f1, f2, f3)

👉 Returns:

- `CompletableFuture<Void>`
- Completes when **ALL tasks finish**

🔥 Interview Line

> “allOf is used to synchronize multiple async tasks and proceed only after all complete.”

2. `CompletableFuture` vs `parallelStream()` (VERY IMPORTANT)

`parallelStream()`

long sum = Arrays.stream(arr).parallel().asLongStream()..sum();

🔹 `CompletableFuture`

- Manual control over:
  - threads
  - task splitting
  - Execution

⚔️ Comparison Table

Feature	parallelStream	CompletableFuture
Ease	✅ Very easy	❌ More code
Control	❌ Low	✅ High
Thread Pool	Common ForkJoinPool	Custom pool
Debugging	❌ Hard	✅ Easier
Composition	❌ Limited	✅ Powerful
Best for	CPU-bound simple ops	Complex async workflows

🔥 When to Use What

✅ Use `parallelStream()` when:

- Simple operations
- Stateless computation
- CPU-bound tasks

✅ Use `CompletableFuture` when:

- Need custom thread pool
- Need chaining (`thenApply`)
- Need error handling
- Need multiple async dependencies

💣 Interview Trap

❓ “Which is better?”

👉 WRONG answer:

> “parallelStream is always better”

✅ Correct answer:

> “parallelStream is simpler for straightforward parallel processing, but CompletableFuture provides better control, composition, and custom execution management.”

⚡ **Real Interview Scenario**

Q: Process 3 APIs in parallel and combine result

👉 Expected:

CompletableFuture.allOf(...)

❌ NOT:

parallelStream()

🚀 Final Takeaway

- `parallelStream()` = **quick & simple**
- `CompletableFuture` = **powerful & scalable**
- `allOf()` = **sync multiple async tasks**

Parallel Processing:

10. Parallel File Processing / Log Processing

Problem:
Process large files concurrently

Concepts:

- **Thread pool**
- **Chunking**
- **I/O + CPU balance**

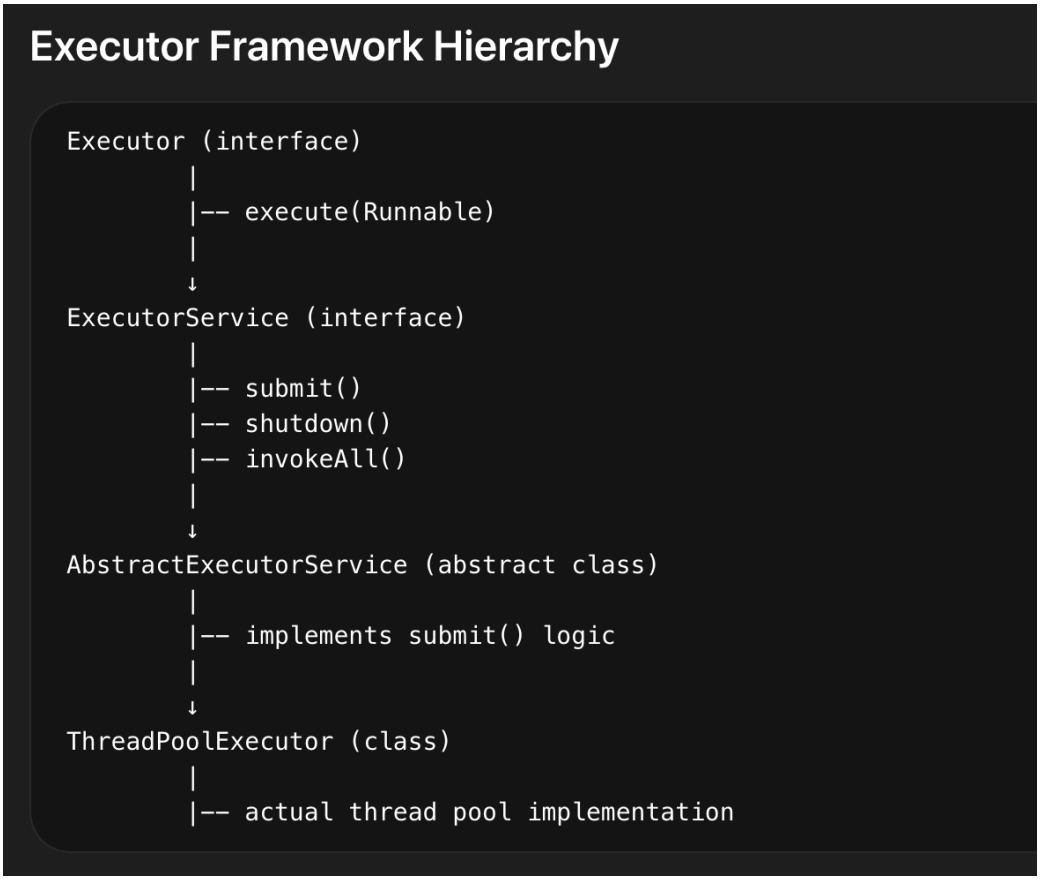
Java Code (Thread Pool + Callable)

```
import java.io.*;
import java.util.*;
import java.util.concurrent.*;

class FileChunkTask implements Callable<Integer> {

    private List<String> lines;

    public FileChunkTask(List<String> lines) {
        this.lines = lines;
    }

    @Override
    public Integer call() {
        int count = 0;

        for (String line : lines) {
            // Example: count lines containing "ERROR"
            if (line.contains("ERROR")) {
                count++;
            }
        }

        return count;
    }
}

public class Main {
```

```
    public static void main(String[] args) throws Exception {

        String filePath = "large_log.txt";

        int numThreads = Runtime.getRuntime().availableProcessors();
        ExecutorService executor = Executors.newFixedThreadPool(numThreads);

        List<Future<Integer>> futures = new ArrayList<>();

        int chunkSize = 1000; // lines per chunk

        BufferedReader reader = new BufferedReader(new FileReader(filePath));

        List<String> chunk = new ArrayList<>();
        String line;

        while ((line = reader.readLine()) != null) {
            chunk.add(line);

            if (chunk.size() == chunkSize) {
                futures.add(executor.submit(new FileChunkTask(new ArrayList<>(chunk))));
                chunk.clear();
            }
        }

        // Remaining lines
        if (!chunk.isEmpty()) {
            futures.add(executor.submit(new FileChunkTask(chunk)));
        }

        reader.close();

        int totalCount = 0;

        for (Future<Integer> future : futures) {
            totalCount += future.get();
        }

        executor.shutdown();

        System.out.println("Total ERROR lines: " + totalCount);
    }
}
```

ExecutorService executor = Executors

Executor Framework Hierarchy



ExecutorService executor = Executors.newFixedThreadPool(3);

```
executor.submit(() -> {
    System.out.println("Task executed by: " + Thread.currentThread().getName());
});
```

ExecutorService executor = Executors.newCachedThreadPool();
executor.submit(() -> System.out.println("Working..."));
executor.shutdown(); // Initiates an orderly shutdown

// Custom Thread Pool

ThreadPoolExecutor customPool = new **ThreadPoolExecutor**(
    int corePoolSize,
    int maximumPoolSize,

```
    long keepAliveTime,
    TimeUnit unit,
    BlockingQueue<Runnable> workQueue
);
```

ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);

```
scheduler.schedule(() -> {
    System.out.println("Executed after 3 seconds!");
}, 3, TimeUnit.SECONDS);
```

//delay execution by 3 seconds

```
scheduledExecutorService.scheduleAtFixedRate(()->{
    System.out.println("hello " + System.currentTimeMillis()/1000);
},1,1000, TimeUnit.MILLISECONDS);


scheduledExecutorService.scheduleWithFixedDelay(()->{
     System.out.println("hello " + System.currentTimeMillis()/1000);
},1,1000, TimeUnit.MILLISECONDS);
```

Mimicking the scheduleWithFixedDelay:

```
public class Main {
    public static void main(String[] args) throws ExecutionException,
InterruptedException {
        int n =5;
        ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(1);

        scheduler.scheduleWithFixedDelay(() -> {
            System.out.println("Start: " + System.currentTimeMillis()/1000);

            try {
                Thread.sleep(2000); // simulate work
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
```

```
            System.out.println("End: " + System.currentTimeMillis()/1000);
        }, 0, 3, TimeUnit.SECONDS);

    }

}
```

Meaning

- Task takes: 2 sec
- Delay: 3 sec
- Next execution starts after: 2 + 3 = 5 sec

Mimicking the scheduleAtFixedRate:

Case A: Single thread pool

newScheduledThreadPool(1)

- No piling up
- Tasks run back-to-back
- Scheduler tries to "catch up"

Behavior:

No overlap, but no gap either

Case B: Multiple threads

newScheduledThreadPool(5)

- YES — tasks can pile up
- Multiple executions run in parallel

Behavior:

Thread1 → task1
Thread2 → task2 (scheduled at t=3)
Thread3 → task3 (scheduled at t=6)

🔥 This can cause:

- CPU spike
- Memory pressure
- System instability

```
public class Main {
   public static void main(String[] args) throws ExecutionException,
InterruptedException {
       int n =5;
       ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(2);

       scheduler.scheduleAtFixedRate(() -> {
           System.out.println("Start: " + System.currentTimeMillis()/1000);

           try {
               Thread.sleep(5000); // simulate work
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
           }

           System.out.println("End: " + System.currentTimeMillis()/1000);
       }, 0, 3, TimeUnit.SECONDS);

   }

}
```

Case 1: Single Thread (Most Common & Safe)

✅ Behavior

- **Tasks run one by one**
- **❌ No overlap**
- **✅ Predictable execution**
- **✅ No race conditions (if shared state)**

📌 Use this when:

- **Periodic jobs (polling, cleanup, cron-like)**
- **You want strict order**
- **You want to avoid overlap completely**

👉 **Example:**

- **Log cleanup**
- **Cache refresh**
- **Health checks**

🔥 Case 2: Multiple Threads

ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(5);

✅ Behavior

- **Multiple tasks can run in parallel**
- **⚠️ Overlap possible (especially with `scheduleAtFixedRate`)**
- **⚡ Better throughput**

📌 Use this when:

- **Tasks are independent**
- **Tasks are long-running**
- **You want parallel execution**

👉 **Example:**

- **Processing multiple independent jobs**
- **Parallel API calls**
- **Batch processing**

Monitoring and management facilities

```
System.out.println("Active Threads: " + executor.getActiveCount());
System.out.println("Queued Tasks: " + executor.getQueue().size());
```

Lifecycle control and graceful shutdown capabilities

executor.shutdown();

Feature	execute()	submit()
Interface	Executor	ExecutorService
Return type	void	Future
Accepts	Runnable	Runnable / Callable
Result retrieval	No	Yes
Exception handling	Direct	Stored in Future

Position in Executor Framework

```
Executor (interface)
        ↓
ExecutorService (interface)
        ↓
AbstractExecutorService (abstract class)
        ↓
ThreadPoolExecutor (class)

Executors (utility class)
        ↓
Creates ThreadPoolExecutor instances
```

So:

```
Executors → factory class
ThreadPoolExecutor → actual implementation
```

Thread Executors :

Executor Framework Hierarchy:

Executor (interface)

```
│
├── void execute(Runnable command)
│
└── ExecutorService (interface) extends Executor
    │
    ├── void shutdown()
    ├── List<Runnable> shutdownNow()
    ├── boolean isShutdown()
    ├── boolean isTerminated()
    ├── boolean awaitTermination(long timeout, TimeUnit unit)
    │
    ├── <T> Future<T> submit(Callable<T> task)
    ├── Future<?> submit(Runnable task)
    ├── <T> Future<T> submit(Runnable task, T result)
    │
    ├── <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)
    ├── <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks,
    │                         long timeout, TimeUnit unit)
    │
    ├── <T> T invokeAny(Collection<? extends Callable<T>> tasks)
    ├── <T> T invokeAny(Collection<? extends Callable<T>> tasks,
    │                 long timeout, TimeUnit unit)
    │
    └── ScheduledExecutorService (interface) extends ExecutorService
        │
        ├── ScheduledFuture<?> schedule(Runnable command,
        │                         long delay, TimeUnit unit)
        │
        ├── <V> ScheduledFuture<V> schedule(Callable<V> callable,
        │                           long delay, TimeUnit unit)
        │
        ├── ScheduledFuture<?> scheduleAtFixedRate(Runnable command,
        │                                 long initialDelay, long period, TimeUnit unit)
        │
        └── ScheduledFuture<?> scheduleWithFixedDelay(Runnable command,
                                           long initialDelay,
                                           long delay, TimeUnit unit)
```

Core Implementations:

ThreadPoolExecutor (class) **extends AbstractExecutorService**
(AbstractExecutorService implements **ExecutorService)**

```
│
├── execute()
├── submit()
├── shutdown()
├── shutdownNow()
├── beforeExecute()
├── afterExecute()
├── terminated()
│
└── ScheduledThreadPoolExecutor (class) extends ThreadPoolExecutor  implements
      ScheduledExecutorService
```

Factory Methods (Executors class):

```
Executors (utility class)
│
├── newFixedThreadPool(int nThreads)
├── newCachedThreadPool()
├── newSingleThreadExecutor()
├── newScheduledThreadPool(int corePoolSize)
├── newSingleThreadScheduledExecutor()
├── newWorkStealingPool()
```

Supporting Interfaces / Classes:

```
Future<T>
│
├── T get()
├── T get(long timeout, TimeUnit unit)
├── boolean cancel(boolean mayInterruptIfRunning)
├── boolean isDone()
├── boolean isCancelled()

Callable<V>
│
└── V call()

Runnable
│
```

└── void run()

Interview Notes

- `Executor` → only **fire-and-forget**
- `ExecutorService` → **lifecycle + result handling**
- `submit()` vs `execute()`:
  - `execute()` → no result
  - `submit()` → returns `Future`
- `invokeAll()` → wait for ALL tasks
- `invokeAny()` → return FIRST completed
- `ScheduledExecutorService` → replaces `Timer`
- `shutdown → graceful shutdown`
- `shutdownNow → interrupts`
- `beforeExecute / afterExecute : Hooks for logging / monitoring`

`Executors:`

`Executors` is a **utility class** in `java.util.concurrent` that provides **factory methods to create different types of thread pools**.
it **creates objects that implement `ExecutorService`**, usually `ThreadPoolExecutor`.

```
public static ExecutorService newFixedThreadPool(int nThreads) {
   return new ThreadPoolExecutor(nThreads, nThreads,
                                 0L, TimeUnit.MILLISECONDS,
                                 new LinkedBlockingQueue<Runnable>());
}
```

```
public static ExecutorService newFixedThreadPool(int nThreads, ThreadFactory
threadFactory) {
   return new ThreadPoolExecutor(nThreads, nThreads,
                                 0L, TimeUnit.MILLISECONDS,
                                 new LinkedBlockingQueue<Runnable>(),
                                 threadFactory);
}
```

```
public static ExecutorService newCachedThreadPool() {
   return new ThreadPoolExecutor(0, Integer.MAX_VALUE,
                                 60L, TimeUnit.SECONDS,
                                 new SynchronousQueue<Runnable>());
}
```

```
public static ExecutorService newCachedThreadPool(ThreadFactory threadFactory)
{
   return new ThreadPoolExecutor(0, Integer.MAX_VALUE,
                                 60L, TimeUnit.SECONDS,
                                 new SynchronousQueue<Runnable>(),
                                 threadFactory);
}
```

Executors creating thread pool using ScheduledThreadPoolExecutor:

```
public static ScheduledExecutorService newScheduledThreadPool(int corePoolSize)
{
   return new ScheduledThreadPoolExecutor(corePoolSize);
}
```

```
public static ScheduledExecutorService newScheduledThreadPool(
       int corePoolSize, ThreadFactory threadFactory) {
   return new ScheduledThreadPoolExecutor(corePoolSize, threadFactory);
}
```

What is ThreadPoolExecutor?

`ThreadPoolExecutor` is the **core implementation of ExecutorService** that manages:

- **Thread creation & reuse**
- **Task queueing**
- **Worker lifecycle**
- **Rejection handling**

It is the **actual engine behind** `Executors.newFixedThreadPool()`, `newCachedThreadPool()` etc.

ThreadPoolExecutor customPool = new **ThreadPoolExecutor**(
int corePoolSize,
int maximumPoolSize,
long keepAliveTime,
TimeUnit unit,
BlockingQueue<Runnable> workQueue
);

Example :
ThreadPoolExecutor executor = new ThreadPoolExecutor(2, 4, 1, TimeUnit.SECONDS, new LinkedBlockingQueue<>(2));

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue)
```

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory)
```

```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          RejectedExecutionHandler handler)
```

```java
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue,
                          ThreadFactory threadFactory,
                          RejectedExecutionHandler handler)
```

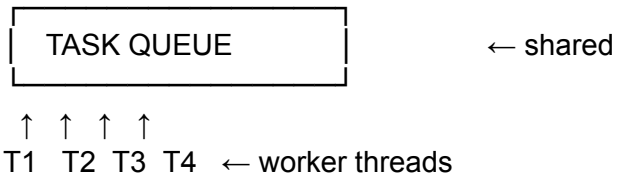
1. Core Parameters

ThreadPoolExecutor(
int ***corePoolSize****,*
int ***maximumPoolSize****,*
long ***keepAliveTime****,*
TimeUnit ***unit****,*
BlockingQueue<Runnable> ***workQueue****,*
ThreadFactory ***threadFactory****,*
RejectedExecutionHandler ***handler***
)

Meaning

- **corePoolSize** → min threads always alive
- **maximumPoolSize** → max threads allowed
- **keepAliveTime** → idle thread timeout
- **workQueue** → where tasks wait
- **handler** → rejection policy

The queue is shared by all worker threads in a `ThreadPoolExecutor`.



Execution Flow (Step-by-step)

When `execute(task)` is called:

1. If `currentThreads < corePoolSize`
   → create new thread
2. Else → put task in queue
3. If queue full & `threads < maxPoolSize`
   → create new thread
4. Else → reject task (via handler)

Worker Thread Concept

- Each thread is a **Worker**
- Worker:
  - runs tasks
  - fetches next task from queue
- Uses **runWorker() loop internally**

Rejection Policies

Built-in:

- `AbortPolicy` → throws exception
- `CallerRunsPolicy` → runs in caller thread
- `DiscardPolicy` → silently drops
- `DiscardOldestPolicy` → removes oldest

Queue Types Impact Behavior

Queue	Behavior
`LinkedBlockingQueue`	queue first, then scale
`SynchronousQueue`	no queue → direct thread

`ArrayBlockingQueue` bounded queue

Internal Working

```
if (workers < corePoolSize)
    addWorker(task)
else if (queue.offer(task))
    // queued
else if (workers < maxPoolSize)
    addWorker(task)
else
    reject(task)
```

Real Interview Traps

Why is newFixedThreadPool dangerous?

- Uses **unbounded queue**
- Can cause **OOM (Out of memory risks)**

Difference: core vs max threads?

- Core → always alive
- Max → burst handling

When threads die?

- Only when:
    - idle > keepAliveTime
    - above above corePoolSize
    - or core timeout enabled

Why BlockingQueue?

- Decouples producer & consumer
- Controls load

ScheduledThreadPoolExecutor

ScheduledThreadPoolExecutor:

`ScheduledThreadPoolExecutor` is a concrete implementation of **ScheduledExecutorService** used to **schedule tasks to run after a delay or periodically**.

It extends:

- ThreadPoolExecutor and implements:
- ScheduledExecutorService

Why do we need it?

It solves problems like:

- Run a task **after some delay**
- Run a task **repeatedly (cron-like behavior)**
- Replace old Timer (which has limitations)

Key Features

- Supports **delayed execution**
- Supports **periodic execution**
- Uses a **thread pool (not single thread like Timer)**
- Handles **exceptions better**
- Uses internal **DelayQueue**

```
public ScheduledThreadPoolExecutor(int corePoolSize) {
   super(corePoolSize, Integer.MAX_VALUE,
         DEFAULT_KEEPALIVE_MILLIS, MILLISECONDS,
         new DelayedWorkQueue());
}
```

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   ThreadFactory threadFactory) {
   super(corePoolSize, Integer.MAX_VALUE,
         DEFAULT_KEEPALIVE_MILLIS, MILLISECONDS,
         new DelayedWorkQueue(), threadFactory);
}
```

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   RejectedExecutionHandler handler) {
   super(corePoolSize, Integer.MAX_VALUE,
         DEFAULT_KEEPALIVE_MILLIS, MILLISECONDS,
         new DelayedWorkQueue(), handler);
```

```
public ScheduledThreadPoolExecutor(int corePoolSize,
                                   ThreadFactory threadFactory,
                                   RejectedExecutionHandler handler) {
   super(corePoolSize, Integer.MAX_VALUE,
         DEFAULT_KEEPALIVE_MILLIS, MILLISECONDS,
         new DelayedWorkQueue(), threadFactory, handler);
```

Important Methods

1. `schedule()` → Run once after delay

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

scheduler.schedule(() -> {
   System.out.println("Task executed after 3 seconds");
}, 3, TimeUnit.SECONDS);
```

2. `scheduleAtFixedRate()` → Fixed interval

```
scheduler.scheduleAtFixedRate(() -> {
   System.out.println("Running every 2 seconds");
}, 0, 2, TimeUnit.SECONDS);
```

Behavior:

- Starts every **2 sec from start time**
- If task takes longer → **next execution may overlap**

---

3. `scheduleWithFixedDelay()` → Delay after completion

```
scheduler.scheduleWithFixedDelay(() -> {
   System.out.println("Running with delay");
}, 0, 2, TimeUnit.SECONDS);
```

Behavior:

- Waits **2 sec AFTER task finishes**
- No overlap → safer for long tasks

FixedRate vs FixedDelay :

Feature	scheduleAtFixedRate	scheduleWithFixedDelay
Timing	Based on start time	Based on end time
Overlap	Possible	Not possible
Use case	Monitoring, metrics	Background jobs

Internal Working

- Uses DelayQueue
- Each task = `ScheduledFutureTask`
- Tasks sorted by execution time
- Worker threads pick tasks when delay expires

Flow:

Task → DelayQueue → Worker Thread → Execute

```
import java.util.concurrent.*;

public class SchedulerExample {
    public static void main(String[] args) {

        ScheduledThreadPoolExecutor scheduler =
                new ScheduledThreadPoolExecutor(2);

        scheduler.scheduleAtFixedRate(() -> {
            System.out.println(Thread.currentThread().getName()
                    + " Running task " + System.currentTimeMillis());
        }, 0, 2, TimeUnit.SECONDS);

        try {
            Thread.sleep(10000);
        } catch (InterruptedException e) {}

        scheduler.shutdown();
    }
}
```

1. Why not use `Timer`?

- Single thread → blocking issue
- If one task fails → all stop

2. Difference between `ScheduledThreadPoolExecutor` and `ThreadPoolExecutor`?

Feature	ThreadPoolExecutor	ScheduledThreadPoolExecutor
Scheduling	❌	✅
Delay support	❌	✅
Queue	**BlockingQueue**	**DelayQueue**

3. What happens if task throws exception?

For periodic tasks:

- **Task stops executing further**

---

4. Can tasks overlap?

- `scheduleAtFixedRate` → **YES**
- `scheduleWithFixedDelay` → **NO**

5. What is core pool size here?

- **Number of threads handling scheduled tasks**
- **Fixed (no max pool like ThreadPoolExecutor)**

When to Use in Real Systems

- Health check scheduler (NMS systems 🔥)
- Polling APIs every few seconds
- Cache refresh
- Retry mechanisms
- Heartbeat signals

Example :

```
import java.util.concurrent.*;

public class Main {

    public static void main(String[] args) throws InterruptedException {

        ScheduledThreadPoolExecutor scheduler =
                new ScheduledThreadPoolExecutor(2); // 2 threads

        scheduler.setRemoveOnCancelPolicy(true); // good practice

        scheduler.scheduleAtFixedRate(() -> {
            System.out.println("Running at: " + System.currentTimeMillis() +
                    " | Thread: " + Thread.currentThread().getName());

            try {
                Thread.sleep(2000); // simulate work
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

        }, 0, 3, TimeUnit.SECONDS);

        Thread.sleep(15000);

        System.out.println("Shutting down...");
        scheduler.shutdown();
    }
}
```

Scheduler

Implement the scheduler which will update my object , let' say device object every 10 minutes

```
import java.util.concurrent.*;

class Device {
    private final String id;
    private volatile String status;
    private volatile long lastUpdated;

    public Device(String id) {
        this.id = id;
        this.status = "INIT";
        this.lastUpdated = System.currentTimeMillis();
    }

    public synchronized void updateStatus() {
        this.status = "UPDATED@" + System.currentTimeMillis();
        this.lastUpdated = System.currentTimeMillis();

        System.out.println(Thread.currentThread().getName() +
                " -> Device " + id + " updated at " + lastUpdated);
    }
}


public class Main {

    public static void main(String[] args) throws InterruptedException {
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(4);

        int numberOfDevices = 5;

        // Schedule devices
        for (int i = 1; i <= numberOfDevices; i++) {
            Device device = new Device("D" + i);

            scheduler.scheduleWithFixedDelay( () -> {
                    try {
                        device.updateStatus();
                    } catch (Exception e) {
                        System.out.println("Error: " + e.getMessage());
                    }
```

```
            }, 0, 10, TimeUnit.MINUTES);
        }

        Thread.sleep(30000); // run for 30 seconds

        System.out.println("Shutting down...");
        scheduler.shutdown();
    }
}
```

For multiple devices, above approach is not good:

🔹 Use Single Scheduler + Worker Pool

👉 Idea:

- Scheduler triggers periodically
- Worker pool processes devices in parallel

✅ Design

Scheduler → triggers every X time
↓
Submit 100 tasks to worker thread pool
↓
Parallel execution

🔥 Why this is BEST

✅ 1. Controlled concurrency

- Only **10 threads working**
- Not 100 threads

✅ 2. No scheduling drift

- One scheduler controls timing
- Not dependent on thread availability

✅ 3. Scalable

- Works for:
  - 100 devices ✅
  - 10,000 devices ✅

✅ 4. No starvation

- No device waits because scheduler thread busy

```
1
2 ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
3 ExecutorService workerPool = Executors.newFixedThreadPool(10); // tune this
4
5 List<Device> devices = getDevices(); // 100 devices
6
7 scheduler.scheduleWithFixedDelay(() -> {
8
9     for (Device device : devices) {
10        workerPool.submit(() -> {
11            try {
12                device.updateStatus();
13            } catch (Exception e) {
14                System.out.println("Error: " + e.getMessage());
15            }
16        });
17    }
18
19 }, 0, 10, TimeUnit.MINUTES);
20
```

⚠️ Important Enhancement (VERY IMPORTANT)

🔹 Use batching (for large scale)

Instead of 1 device per task:

```
List<List<Device>> batches = partition(devices, 10);

for (List<Device> batch : batches) {
   workerPool.submit(() -> {
      for (Device d : batch) {
         d.updateStatus();
      }
   });
}
```

💣 Interview Answer (Perfect)

> “Instead of scheduling 100 independent tasks, I would use a single scheduler to trigger execution and a worker thread pool to process devices in parallel. This provides better scalability, controlled concurrency, and avoids scheduling drift.”

DelayQueue:

DelayQueue:

`DelayQueue` is a **blocking queue** in Java that holds elements **only after a delay has expired**.

It is part of java.util.concurrent

Core Idea

- Elements must implement:
  👉 Delayed
- Queue behavior:
  - You can insert anytime
  - You can **only take when delay = 0**
  - Elements are ordered by **remaining delay (min first)**

🧠 Key Properties

Feature	Description
Type	Unbounded Blocking Queue
Ordering	Based on delay time
Thread-safe	Yes
Blocking nature	`take()` waits until element expires
Internal DS	Priority Queue (Min Heap)

Important Methods

```
void put(E e)
E take()          // Blocks until delay expires
E poll()          // Returns null if not expired
E peek()          // Check head without removing
int size()
```

How It Works Internally

1. Uses **PriorityQueue (min heap)**
2. Element with **smallest delay** is at top
3. `take()`:
    - If delay > 0 → thread waits
    - If delay = 0 → returns element

Real Use Cases

1. Task Scheduling (Without Scheduler)

- Retry after delay
- Backoff strategies

2. Cache Expiry (TTL)

- Remove elements after expiry

3. Message Delay System

- Delayed processing (like Kafka retry)

AbstractExecutorService:

`AbstractExecutorService:`

AbstractExecutorService is a core abstract class in Java's concurrency framework that provides default implementations for many methods of the ExecutorService interface.

It acts as a base class so that you don't have to implement everything from scratch when creating custom executors.

You only need to implement:

protected <T> RunnableFuture<T> newTaskFor(Callable<T> callable)
protected <T> RunnableFuture<T> newTaskFor(Runnable runnable, T value)
void execute(Runnable command)

🔹 Important Methods (with behavior)

1. submit()

Future<?> submit(Runnable task)
Future<T> submit(Callable<T> task)
Future<T> submit(Runnable task, T result)

✔ Converts task → `RunnableFuture`
✔ Calls `execute()` internally

---

2. invokeAll()

List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)

✔ Executes all tasks
✔ Waits for ALL to complete
✔ Returns list of Futures

---

3. invokeAny()

T invokeAny(Collection<? extends Callable<T>> tasks)

✔ Executes multiple tasks
✔ Returns result of **first successfully completed task**
✔ Cancels others

---

4. newTaskFor() (VERY IMPORTANT 🔥)

protected <T> RunnableFuture<T> newTaskFor(Callable<T> callable)

✔ Wraps task into `FutureTask`
✔ Used internally by `submit()`

🔹 **Internal Flow of submit()**

submit(task)
↓
newTaskFor(task) → creates FutureTask
↓
execute(futureTask)
↓
ThreadPool runs it
↓
FutureTask stores result

Exceptions

Exceptions

Checked vs Unchecked Exceptions in Java :

Checked Exceptions :
Checked exceptions are exceptions that must be either declared in the method signature using throws or handled using try-catch.
They are checked at compile-time.
Examples:
IOException (e.g., file not found)
SQLException (e.g., database connection failure)
InterruptedException (e.g., thread interruption)

Unchecked Exceptions :
Unchecked exceptions are runtime errors that do not require explicit handling.
They are checked at runtime, meaning they occur due to logical errors in the program.
Examples:
NullPointerException (e.g., calling a method on null)
ArrayIndexOutOfBoundsException (e.g., accessing an invalid array index)
ArithmeticException (e.g., division by zero)

Exceptions in Future:

1. One task failure does NOT stop others

- All tasks run independently

```
long total = 0;

for (Future<Long> f : futures) {
    try {
        total += f.get();
    } catch (ExecutionException e) {
        System.out.println("Task failed: " + e.getCause());
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

What happens if one task throws an exception?

Suppose:

```
for (int i = 0; i < 5; i++) {
   futures.add(executor.submit(() -> {
       if (ThreadLocalRandom.current().nextInt(5) == 0) {
           throw new RuntimeException("Failure");
       }
       return 10L;
   }));
}
```

Case 1: While submitting tasks

```
futures.add(executor.submit(...));
```

- All tasks are submitted normally
- Even if one task fails → **others are NOT affected**
- Program continues

Case 2: While collecting results (IMPORTANT)

```
for (Future<Long> f : futures) {
   total += f.get();
}
```

What happens?

- If ANY task failed →
  `f.get()` throws **`ExecutionException`**

👉 But only for THAT specific future

Important Behavior

- Loop **stops immediately** if exception not handled
- Remaining futures are **never processed** ❌

Using `CompletableFuture`:

```
long total = futures.stream()
   .map(f -> {
      try {
         return f.get();
      } catch (Exception e) {
         return 0L; // fallback
      }
   })
   .mapToLong(Long::longValue)
   .sum();
```

Reentrant Locks:

Reentrant Locks:

1. lock()

2. unlock()

Releases the lock.

3. tryLock()

Attempts to acquire lock **without blocking**.

```
if (lock.tryLock()) {
   try {
       // critical section
   } finally {
       lock.unlock();
   }
}
```

4. tryLock(long time, TimeUnit unit)

Waits for a specified time.

When the same thread acquires the lock again:

- The **hold count increases**

Example:

```
lock.lock() → hold count = 1
lock.lock() → hold count = 2
lock.unlock() → hold count = 1
lock.unlock() → hold count = 0 (lock released)
```

Only when **hold count becomes 0** is the lock released and available for other threads.

3. Fair Lock

new ReentrantLock(true)

Provides fairness:

FIFO style

Lock Internals

ReentrantLock uses:

CAS + AQS(AbstractQueuedSynchronizer)

Which One Should You Use?

Use synchronized

When:

- simple locking needed

- basic thread safety
- cleaner code

Use Lock

When you need:

- tryLock()
- timeout
- interruptibility
- fairness
- advanced concurrency control

The real differentiator is:

Lock gives CONTROL

synchronized gives SIMPLICITY

That is the core difference.

Lock

You manually control locking behavior.

```
lock.lock();
try {
} finally {
   lock.unlock();
}
```

Now you can:

- try acquiring lock
- timeout
- interrupt
- implement fairness
- use multiple conditions

It is: powerful but verbose

CopyOnWriteArrayList:

CopyOnWriteArrayList:

2. CopyOnWriteArrayList:

CopyOnWriteArrayList is a thread-safe variant of ArrayList that creates a fresh copy of the underlying array for every modification operation. This unique approach ensures thread safety while providing non-blocking read operations, making it particularly well-suited for specific concurrency scenarios. 🔒

Both ArrayList and CopyOnWriteArrayList in Java implement the List interface, providing ordered collections of elements. 📚 However, they differ significantly in their handling of concurrent access, particularly in their approach to thread safety, making them suitable for different scenarios. ⚖️

Here's a breakdown of their differences, functionalities, and thread-safety mechanisms: 🧩🔍

How ArrayList works (Internals): 📋

• ArrayList internally uses a dynamic array to store elements. ➕ When the array reaches its capacity, a new larger array is created, and all elements are copied to the new array. 📦

• When you add or remove elements, the array is modified directly, and in multi-threaded environments, this can lead to data corruption if multiple threads modify the array concurrently. ⚠️🧵

• Operations like get(), add(), and remove() directly access or modify the underlying array without any built-in synchronization. 🚫🔐\

• If one thread is iterating through an A
rrayList while another thread modifies it, a ConcurrentModificationException is likely to be thrown, as the iterator detects that the collection has been modified during iteration. 🚨🔁

How CopyOnWriteArrayList Works (Internals): 📝

• CopyOnWriteArrayList maintains an immutable array that is only changed by creating and reassigning a new array instance. 🔁📄

• When a modification operation (add, remove, set) is performed, the entire array is copied to a new array with the modification applied, and the reference to the array is atomically updated to point to the new array. ⚙️🔄

• Read operations (like get(), size(), contains()) operate on the current array reference without any locking, providing non-blocking reads. 🚀👓

• Iterators created from the list work on a snapshot of the array at the time the iterator was created, making them immune to concurrent modifications and eliminating the need to throw ConcurrentModificationException. 🛡️🧭

CopyOnWriteArrayList vs. ArrayList: How Thread Safety Differs (Basic): 🔄🧷

The core difference in thread safety lies in how they manage access when multiple threads interact with the list. 👥🧠

ArrayList 🗒️:

Not thread-safe by default. 🚫 When multiple threads access and modify an ArrayList concurrently, it can lead to data corruption or inconsistent states. 💥 Imagine multiple people trying to modify a document simultaneously without any coordination – the result would be chaos. 📝😵

Solutions for thread safety with ArrayList typically involve external synchronization, such as using Collections.synchronizedList() which essentially locks the entire list during any operation, severely limiting concurrency. 🔒🧍🧍

Example: If one thread is adding elements to an ArrayList while another thread is iterating over it, a ConcurrentModificationException will likely be thrown, or worse, the iteration might miss elements or see partially updated states. ⚠️📉

Completable Future:

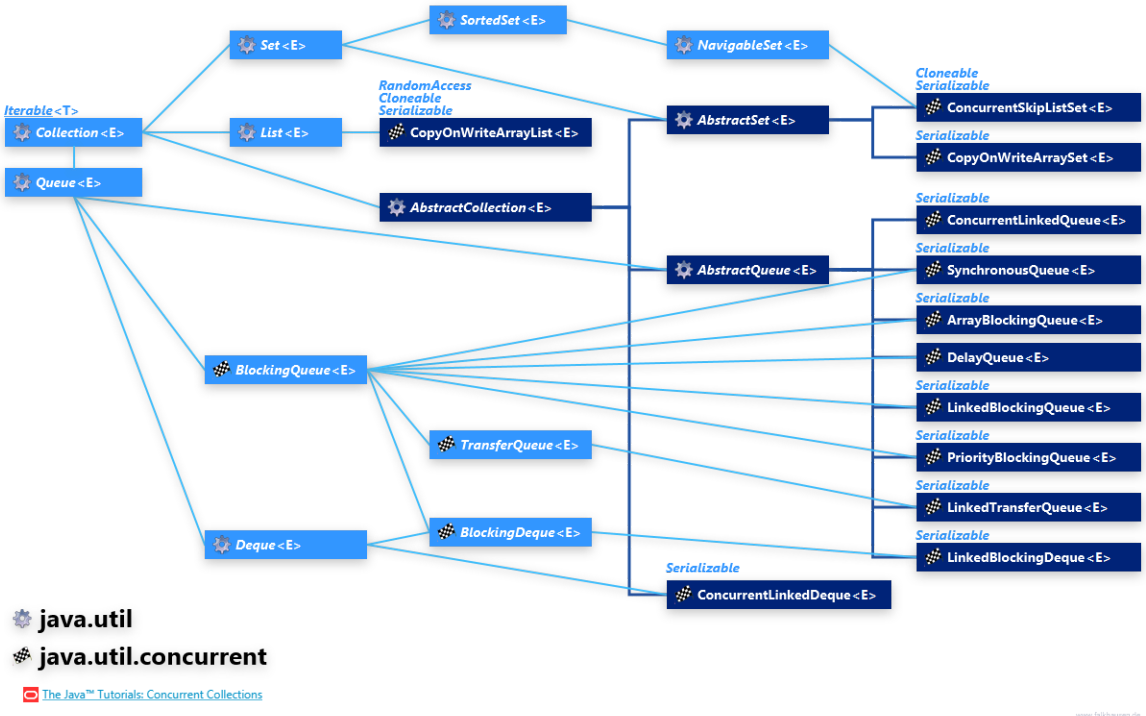
Completable Future:

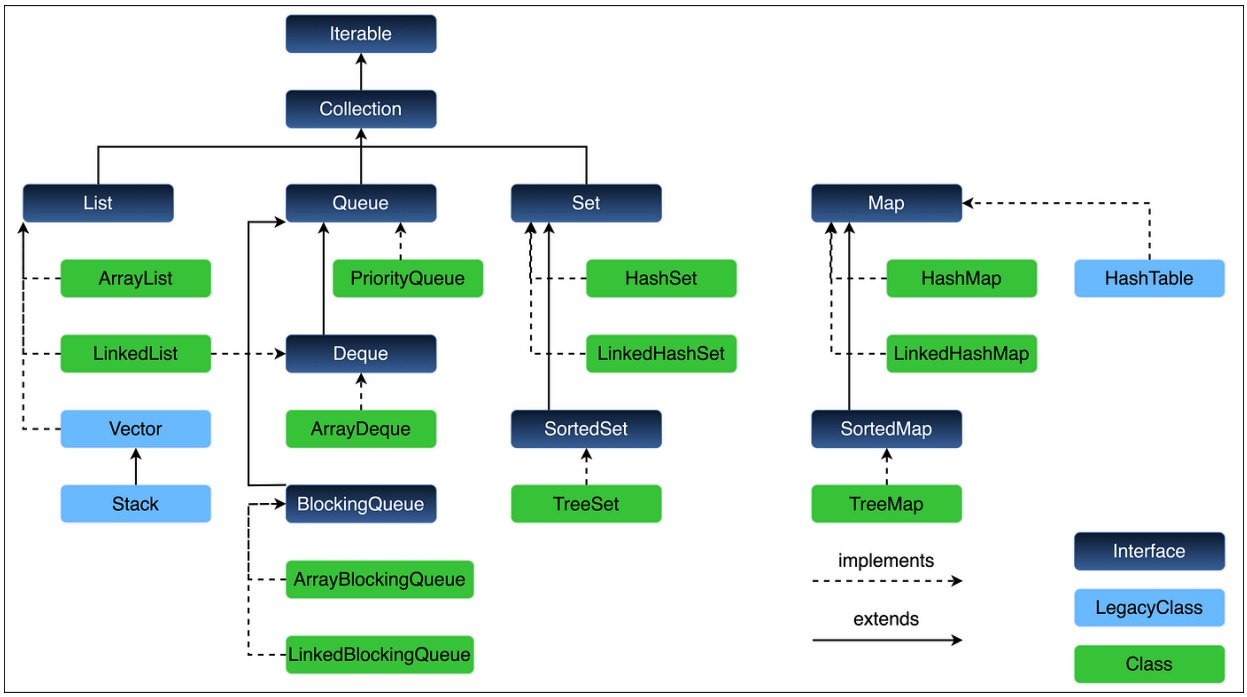
```
import java.util.concurrent.*;

public class CompletableFutureExample {

    public static void main(String[] args) throws Exception {

        ExecutorService executor = Executors.newFixedThreadPool(2);

        // Task 1: Fetch User
        CompletableFuture<String> userFuture = CompletableFuture.supplyAsync(() -> {
            sleep(1000);
            return "User: Kapil";
        }, executor);

        // Task 2: Fetch Orders
        CompletableFuture<String> orderFuture = CompletableFuture.supplyAsync(() -> {
            sleep(1500);
            return "Orders: [Book, Laptop]";
        }, executor);

        // Combine both results
        CompletableFuture<String> combinedFuture =
                userFuture.thenCombine(orderFuture, (user, orders) -> user + " | " + orders);

        // Final result
        String result = combinedFuture.get();
        System.out.println(result);

        executor.shutdown();
    }

    private static void sleep(int millis) {
        try {
            Thread.sleep(millis);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

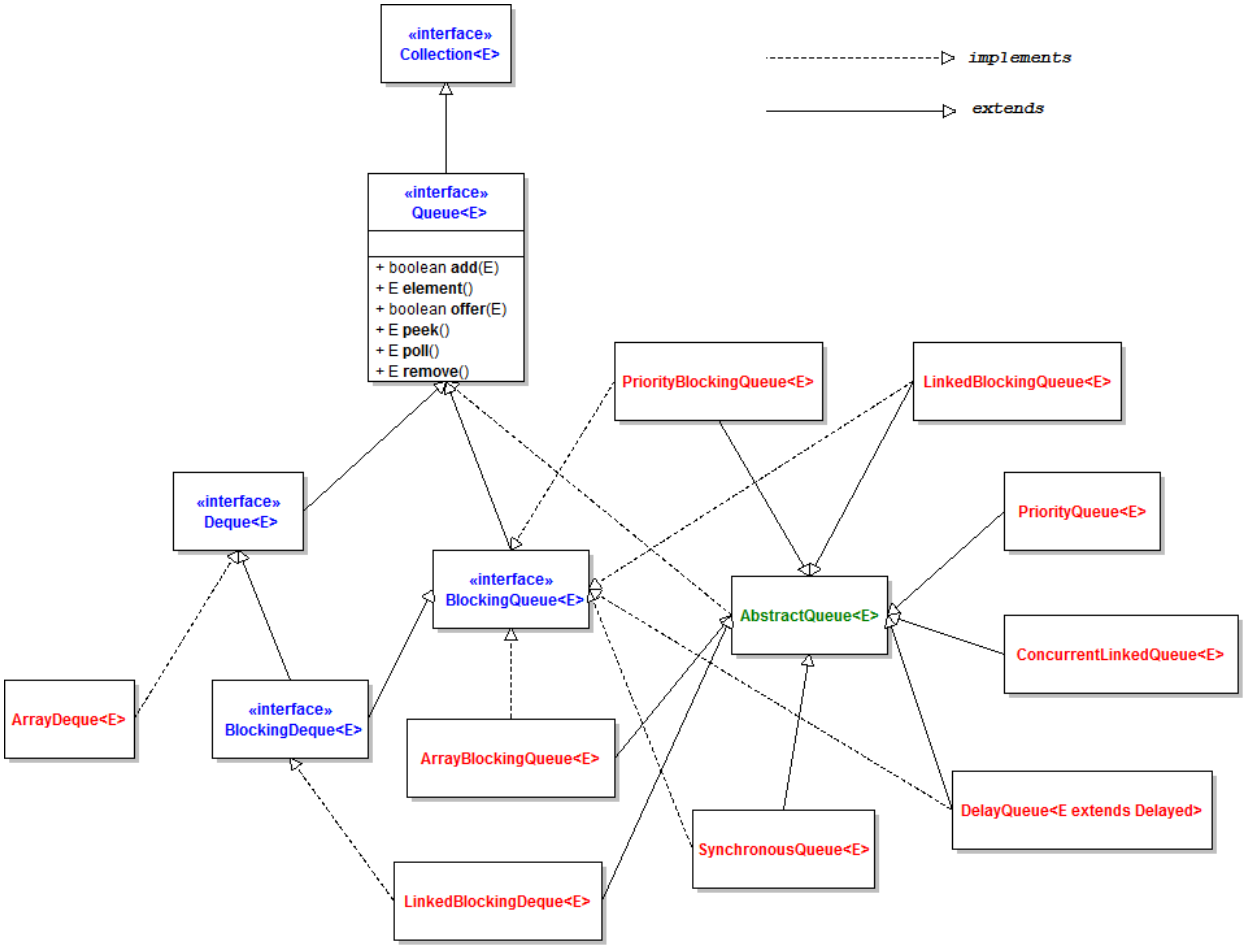
⚡ Ultra-Short Revision

- `supplyAsync()` → async task with return
- `runAsync()` → async task no return
- `thenApply()` → transform
- `thenCompose()` → chain async
- `thenCombine()` → merge 2 futures
- `allOf()` → wait all

Hierarchy


Iterable <T>
Collection <E>
Queue <E>
Set <E>
List <E>
SortedSet <E>
RandomAccess
Cloneable
Serializable
CopyOnWriteArrayList <E>
AbstractCollection <E>
BlockingQueue <E>
TransferQueue <E>
Deque <E>
BlockingDeque <E>
NavigableSet <E>
AbstractSet <E>
AbstractQueue <E>
ConcurrentLinkedDeque <E>
Cloneable
Serializable
ConcurrentSkipListSet <E>
Serializable
CopyOnWriteArraySet <E>
ConcurrentLinkedQueue <E>
SynchronousQueue <E>
ArrayBlockingQueue <E>
DelayQueue <E>
LinkedBlockingQueue <E>
PriorityBlockingQueue <E>
LinkedTransferQueue <E>
LinkedBlockingDeque <E>
java.util
java.util.concurrent
The Java™ Tutorials: Concurrent Collections


Iterable
Collection
List
Queue
Set
Map
ArrayList
LinkedList
Vector
Stack
PriorityQueue
Deque
ArrayDeque
BlockingQueue
ArrayBlockingQueue
LinkedBlockingQueue
HashSet
LinkedHashSet
SortedSet
TreeSet
HashMap
LinkedHashMap
SortedMap
TreeMap
HashTable
implements
extends
Interface
LegacyClass
Class


«interface»
Collection<E>
implements
extends
«interface»
Queue<E>
+ boolean add(E)
+ E element()
+ boolean offer(E)
+ E peek()
+ E poll()
+ E remove()
PriorityBlockingQueue<E>
LinkedBlockingQueue<E>
«interface»
Deque<E>
PriorityQueue<E>
«interface»
BlockingQueue<E>
AbstractQueue<E>
ConcurrentLinkedQueue<E>
ArrayDeque<E>
«interface»
BlockingDeque<E>
ArrayBlockingQueue<E>
DelayQueue<E extends Delayed>
SynchronousQueue<E>
LinkedBlockingDeque<E>

NETCONF (Network Configuration Protocol)

NETCONF (Network Configuration Protocol)

NETCONF is a protocol used to **install, modify, and delete configuration** on network devices (routers, switches, optical nodes, etc.) in a **structured and reliable way**.

Defined by

Standardized by the Internet Engineering Task Force (RFC 6241)

Why NETCONF?

Traditional CLI (SSH/Telnet):

- Manual
- Error-prone
- Hard to automate

NETCONF solves this by:

- Using **structured data (XML)**
- Supporting **transactions (commit/rollback)**
- Providing **automation-friendly APIs**

How NETCONF Works

Communication

- Runs over **SSH**
- Default port: **830**

Basic Flow

Client (NMS) ⇄ NETCONF ⇄ Network Device

1. Client (NMS) connects via SSH

2. Exchanges **XML-based RPC messages**
3. Device responds with structured data

Key Operations (RPCs)

1. `<get>`

- Fetch running config or state data

2. `<edit-config>`

- Modify configuration

3. `<copy-config>`

- Copy configs (running → startup)

4. `<delete-config>`

- Delete configuration

5. `<lock>` / `<unlock>`

- Prevent concurrent changes

6. `<commit>`

- Apply changes (important for candidate config)

Datastores (Very Important)

- **running** → active config
- **candidate** → staged config
- **startup** → boot config

Flow:

candidate → commit → running

Example (XML Request)

```
<rpc>
 <get>
  <filter>
    <interfaces/>
  </filter>
 </get>
</rpc>
```

Key Features

- Transactional (commit / rollback)
- Locking mechanism
- Structured data (XML)
- Supports partial updates
- Reliable over SSH

NETCONF vs REST (Modern Comparison)

Feature	NETCONF	REST API
Format	XML	JSON/XML
Transport	SSH	HTTP/HTTPS
Use case	Network devices	Web services

RADIUS:

RADIUS:

The **RADIUS Protocol (Remote Authentication Dial-In User Service)** is a widely used networking protocol for **authentication, authorization, and accounting (AAA)**. It is commonly used by ISPs, enterprise networks, and telecom systems to control access to network resources.

1. What is RADIUS?

RADIUS is a **client-server protocol** where:

- **Client** : Network device (NAS: router, switch, access point)
- **Server** : RADIUS server (validates users)

It works over **UDP**

- Authentication → Port **1812**
- Accounting → Port **1813**

---

2. Core Functions (AAA)

1. Authentication

Verifies **who the user is**

- Username/password
- Certificates
- Tokens

2. Authorization

Determines **what the user can access**

- VLAN assignment
- IP allocation
- Bandwidth limits

3. Accounting

Tracks **user activity**

- Session start/stop
- Data usage
- Time duration

---

3. RADIUS Architecture

User : NAS (Router/Switch/AP) : RADIUS Server : Database

Flow:

1. User tries to connect (WiFi/VPN)
2. NAS sends request to RADIUS server
3. RADIUS validates credentials
4. Access Accept / Reject sent back
5. Accounting logs maintained

---

4. Packet Types

Authentication Packets

- **Access-Request** : Sent by client
- **Access-Accept** : Success
- **Access-Reject** : Failure
- **Access-Challenge** : Additional verification (OTP)

Accounting Packets

- **Accounting-Request**
- **Accounting-Response**

---

5. Key Features

- Centralized authentication

- Scalable for large networks
- Supports multiple authentication methods (PAP, CHAP, EAP)
- Encrypts only password (not full packet)

---

6. Example (WiFi Login)

1. User connects to enterprise WiFi
2. Access Point acts as NAS
3. Credentials sent to RADIUS server
4. Server validates (e.g., Active Directory)
5. User gets access if valid

---

7. Limitations

- Uses **UDP** : unreliable transport
- Only password is encrypted (rest is plaintext)
- Less secure compared to modern protocols

CORBA stands for:

Common Object Request Broker Architecture

CORBA is a standard defined by the Object Management Group that enables communication between distributed objects written in different programming languages and running on different machines.

Key Concept

- Uses an ORB (Object Request Broker)
- ORB acts like middleware that:
    - Finds the remote object
    - Sends request
    - Returns response

Tejas uses : JACORB

Hashmap:

Hashmap:

HashMap internally uses an array of nodes of initial capacity 16.

When we insert a key-value pair, first hashCode() of key is calculated. Then HashMap applies hash spreading to generate optimized hash. Using this hash and capacity, bucket index is calculated.

If bucket is empty, entry is inserted directly. Otherwise collision occurs, and HashMap traverses the linked list/tree in that bucket. It first compares **hash**, then equals() on keys. If key already exists, value is updated; otherwise new node is appended.

HashMap uses load factor (default 0.75). When entries exceed capacity × load factor, resize happens and capacity doubles. During resize, all entries are rehashed because bucket indexes may change.

In Java 8, if collisions in one bucket exceed threshold 8 and capacity is at least 64, linked list converts into red-black tree, improving worst-case complexity from O(n) to O(log n).

Flow:

hashCode()
↓
optimized hash
↓
bucket index
↓
go to bucket
↓
compare node.hash
↓
compare equals()

Implement a Custom Thread Pool

Implement a Custom Thread Pool

Implement a basic thread pool without using ExecutorService.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class CustomThreadPool {
    private final BlockingQueue<Runnable> queue;

    public CustomThreadPool(int size) {
        queue = new LinkedBlockingQueue<>();

        for (int i = 0; i < size; i++) {
            new Worker().start();
        }
    }

    public void submit(Runnable task) {
        queue.offer(task);
    }

    private class Worker extends Thread {
@override
        public void run() {
            while (true) {
                try {
                    Runnable task = queue.take();
                    task.run();
                } catch (InterruptedException e) {}
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        CustomThreadPool pool = new CustomThreadPool(2);
        pool.submit(() -> System.out.println("Task 1"));
        pool.submit(() -> System.out.println("Task 2"));
    }
}
```

Tab 31

Deep copy vs shallow copy:

```
class Address {
    String city;

    Address(String city) {
        this.city = city;
    }
}

class Person implements Cloneable {
    String name;
    Address address;

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }

    // Shallow Copy
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    // Deep Copy
    Person deepCopy() {
        Address newAddress = new Address(this.address.city);
        return new Person(this.name, newAddress);
    }
}

public class Main {
    public static void main(String[] args) throws Exception {

        Address address = new Address("Delhi");

        Person p1 = new Person("Kapil", address);

        // Shallow Copy
        Person p2 = (Person) p1.clone();

        // Deep Copy
        Person p3 = p1.deepCopy();
```

```
        // Change original object's address
        p1.address.city = "Gurgaon";

        System.out.println("Original: " + p1.address.city);
        System.out.println("Shallow Copy: " + p2.address.city);
        System.out.println("Deep Copy: " + p3.address.city);
    }
}
```

Tab 32

Records
: for immutable classes .

Java Memory Model:
Of course! The Java Memory Model defines how threads interact through memory. It ensures visibility of shared variables and establishes rules like "happens-before," ensuring that changes made by one thread are visible to others. This is crucial for writing correct, thread-safe code!

Method Reference :

Of course! A daemon thread is a background thread that doesn't prevent the JVM from exiting. Once all user threads finish, daemon threads are terminated automatically. They're typically used for things like background tasks or cleanup, whereas user threads keep the application alive.

What is the difference between the String.intern() method and a regular string in Java?

The intern() method stores a string in the string pool, ensuring that identical literals share the same reference. It's useful, but not a core interview focus.

Volatile ;

Exactly! The volatile keyword ensures that reads and writes go directly to main memory, making changes visible to all threads immediately. It's perfect for simple flags or state that's shared among threads.

var:

The "var" keyword allows you to declare local variables without specifying the type explicitly. The compiler infers the type from the context, making code more concise. It doesn't change Java’s static typing—you just let the compiler figure it out!

Tab 33

Q.) What is ThreadLocal?
Ans.)